

Experiment 5

Prompt Chaining for a Smart Traffic System

[Submission PDF](#)

[Source folder on GitHub](#)

Prepared 2026-09-29 from the experiment materials on branch
`codex/complete-experiments`.

Source: [Ex05_Prompt_Chaining_Smart_Traffic/README.md](#)

Ex-5: Comparing Prompting Techniques Through Engineering Problem-Solving (Prompt Chaining)

Due: 30 Aug 2026 **Reference repo:** [RaajaThilahr/Ex.No.5](#)

Project: AI-based Smart Traffic System

Chain: Problem -> Requirement Analysis -> Architecture -> Algorithm -> Flowchart -> Python Code -> Testing -> Documentation

Each step's output is pasted into the next prompt (marked `{{previous_output}}`).

Step	Prompt	Output goes to
1	prompts/1_problem.md	outputs/1_problem.md
2	prompts/2_requirements.md	outputs/2_requirements.md
3	prompts/3_architecture.md	outputs/3_architecture.md
4	prompts/4_algorithm.md	outputs/4_algorithm.md
5	prompts/5_flowchart.md	outputs/5_flowchart.md
6	prompts/6_code.md	src/traffic_controller.py
7	prompts/7_testing.md	tests/test_traffic_controller.py
8	prompts/8_documentation.md	outputs/8_documentation.md

A working reference implementation is already in `src/` and `tests/`. Run: `python -m unittest discover -s tests` from this folder. `comparison.md` compares chained vs single-shot prompting.

Source: [Ex05_Prompt_Chaining_Smart_Traffic/outputs/README.md](#)

Paste each chain step's LLM output as `<n>_<step>.md`.

Source: [Ex05_Prompt_Chaining_Smart_Traffic/LAB_RECORD.md](#)

Lab Record: Ex-5: Prompting Through Engineering Problem-Solving

Field	Entry
Name / Reg. No.	_____
Date	_____
Repo	https://github.com/RHUDHRESH/PROMPT_ENGINEERING-

Aim

Solve an engineering problem via a prompt chain (problem to documentation).

Tools / Apparatus

Claude-generated chain artefacts and Python reference implementation; 6 Python unit tests passed in this session

Procedure

Prompts written (see prompts/ or the folder README).

Each prompt run on every tool; raw outputs saved under outputs/ (or the folder's equivalent).

Results scored in comparison.md on: Consistency, code correctness, effort.

Findings summarised below.

Observations

See comparison.md. Add your own scores for any tool you ran yourself. Tools not run are left blank; do not fill them in from memory.

Result

An eight-step prompt chain links requirements through documentation for a smart traffic controller. The reference code passes six Python unit tests. The single-shot comparison is an unmeasured expectation, not an experimental result.

Conclusion

Chaining makes intermediate requirements reviewable and gives a clear place to find drift. Tests still determine code behaviour; the chain does not guarantee correctness. Broader edge cases and a measured single-shot baseline would strengthen the study.

Faculty signature: _____

Source: Ex05_Prompt_Chaining_Smart_Traffic/comparison.md

Prompt chain vs single-shot

Honesty note. The chain column reflects what was actually produced in this folder (outputs/1..8, src/, tests/). The single-shot column is NOT measured: no single-shot run was performed. It is my reasoned expectation of what a prompt like "build a smart traffic system" would typically give. Token/effort figures were not measured for either column.

Aspect	Single prompt ("build a smart traffic system") - EXPECTED, not measured	Prompt chain (8 steps) - observed
Completeness	Likely a code sketch plus a short description; requirements, flowchart and test gaps probably thin or missing	All eight artefacts exist, plus a list of 16 uncovered edge cases
Consistency between artefacts	Fewer artefacts to conflict, but the design is implicit and unstated priorities may drift	The emergency > starved > longest-queue rule and parameters carry from requirements to algorithm, flowchart, code and docs. The flowchart omits the negative-queue check in green_time, a small gap
Code correctness (tests pass?)	Unknown; would need to be run	The reference code passes its 6 unit tests. The tests are shallow: many edge cases (notes in 7_testing_notes.md) are unchecked and some validation gaps exist in the code
Effort / total tokens	Expected lower: one prompt, one response	Not measured; clearly more (8 prompts, each carrying earlier output). Note that here the chain outputs were written after the code, so they are partly retro-fitted to it
Ease of fixing one stage	Harder: change means re-prompting everything	Easier: edit one stage and re-run downstream steps

Conclusion: Chaining gives traceable, reviewable artefacts and a natural place to spot gaps (for example, the missing validation surfaced while writing step 7). Its cost is more effort. It does not by itself guarantee correctness; the tests do that. Since the single-shot numbers are unmeasured expectations, this comparison is qualitative only. A fair test would run both and count requirements covered and tests passed.

Source: [Ex05_Prompt_Chaining_Smart_Traffic/outputs/1_problem.md](#)

Step 1 - Problem statement

Produced by running prompts/1_problem.md (answer written by Claude, in-session).

Context. A 4-way urban intersection (approaches N, E, S, W) is controlled by fixed-time signals. The fixed cycle ignores actual demand, so green time is wasted on empty approaches while other queues grow.

Current issues

- Fixed green splits do not track queue length, especially at peaks.
- Emergency vehicles are delayed by ordinary queues.
- A very busy approach can dominate a naive "longest queue" policy and leave a small approach waiting indefinitely.
- No central visibility of waiting times or throughput.

Stakeholders: drivers and pedestrians, emergency services, city traffic authority, maintenance staff, urban planners.

Goals: allocate green time in proportion to demand within safe bounds; give emergency vehicles immediate priority; guarantee no approach with waiting vehicles is starved.

Measurable success metrics

Metric	Target (vs fixed-time baseline)
Average wait time per vehicle	reduced by at least 20%
Throughput (vehicles/hour)	increased by at least 10%
Emergency response delay at the junction	under 10 s from detection to green
Maximum consecutive skipped cycles for a non-empty approach	at most 3

Source:
[Ex05_Prompt_Chaining_Smart_Traffic/outputs/2_requirements.md](#)

Step 2 - Requirements

Chained from 1_problem.md (its goals and metrics become the requirements below).

ID	Requirement	Priority (MoSCoW)	Test method
FR1	The system shall read the queue length of each of the four approaches (N, E, S, W).	Must	Unit test with a dict of four queue counts; reject other key sets
FR2	Green time shall be proportional to queue length (seconds-per-vehicle factor).	Must	Unit test: green_time(15) = 30 s with factor 2
FR3	Green time shall be clamped to [min_green, max_green].	Must	Unit test at 0, 100 vehicles

ID	Requirement	Priority (MoSCoW)	Test method
FR4	An emergency vehicle on an approach shall pre-empt normal selection and receive the next green.	Must	Unit test: emergency="W" beats N with 50 queued
FR5	An approach with waiting vehicles skipped max_wait_cycles times shall be served before the longest-queue rule applies (emergency still wins).	Must	Unit test: run 5 cycles, small queue is served
FR6	Otherwise the approach with the longest queue shall be served.	Must	Unit test
FR7	Invalid input (missing directions, negative queue, unknown emergency direction, bad bounds) shall raise an error.	Must	Unit tests expecting ValueError
FR8	A dashboard shall show queues, waits and emergency events.	Should	Manual demo / integration test
NFR1	A decision shall be computed in under 50 ms on edge hardware.	Must	Timing benchmark
NFR2	Fail-safe: on sensor or controller failure, fall back to a fixed-time plan.	Must	Fault-injection test
NFR3	Controller core shall use only the Python standard library.	Should	Dependency check
NFR4	Decision logic shall be deterministic (ties resolved in a fixed order).	Should	Repeat-run test
NFR5	Sensor and cloud links shall be authenticated and encrypted.	Should	Security review
FR9	Machine-learning demand forecasting.	Could	Offline evaluation
FR10	Pedestrian phases and multi-intersection coordination.	Won't (this release)	n/a

Source:

[Ex05_Prompt_Chaining_Smart_Traffic/outputs/3_architecture.md](#)

Step 3 - Architecture

Chained from 2_requirements.md (FR1-FR8, NFR1-NFR5).

Components

Component	Responsibility	Requirements
Sensors (loop detectors or camera + vehicle counter)	Report queue length per approach every cycle	FR1
Emergency-vehicle detector (siren/RF preemption receiver)	Report which approach an emergency vehicle is on	FR4
Edge controller (roadside unit)	Runs the decision engine locally; owns the safety fallback	NFR1, NFR2
Decision engine (TrafficController)	<code>next_phase(queues, emergency)</code> and <code>green_time(queue_len)</code>	FR2-FR7
Signal actuators	Convert the chosen direction and green time into signal states with safe amber/all-red intervals	FR2, NFR2
Cloud dashboard	Stores metrics, shows queues, waits, emergency events	FR8

Data flows

Sensors -> edge controller: {N, E, S, W} queue counts each cycle.

Emergency detector -> edge controller: optional direction (bypasses normal polling).

Edge controller -> decision engine: queues plus emergency flag; returns direction, then `green_time(queue[direction])`.

Edge controller -> actuators: (direction, seconds).

Edge controller -> cloud (asynchronous, batched): decisions, skipped counters, emergencies.

Priority inside the decision engine mirrors the requirements: **emergency > starved > longest queue**.

Technology choices

- Edge: single-board computer running Python 3 (stdlib-only core, NFR3).
- Messaging: MQTT over TLS to the cloud (NFR5); local wired serial or GPIO to the signal controller.
- Cloud: time-series store plus a web dashboard.
- Watchdog timer on the edge box triggers the fixed-time fallback if no decision arrives in time (NFR2).

Source: [Ex05_Prompt_Chaining_Smart_Traffic/outputs/4_algorithm.md](#)

Step 4 - Signal-timing algorithm

Chained from 3_architecture.md (decision engine component).

Parameters: `min_green=10`, `max_green=60`, `seconds_per_vehicle=2.0`, `max_wait_cycles=3`. State: `skipped[d]` for `d` in {N,E,S,W}, initially 0.

`green_time(queue_len)`

If `queue_len < 0`, raise an error.

`raw = queue_len * seconds_per_vehicle`.

Return `int(max(min_green, min(max_green, raw)))`.

next_phase(queues, emergency=None)

If the keys of queues are not exactly {N,E,S,W}, raise an error.

If emergency is given:

If it is not one of N,E,S,W, raise an error.

Return serve(emergency).

starved = all d with skipped[d] >= max_wait_cycles AND queues[d] > 0.

If starved is non-empty, return serve(the starved direction with the largest queue).

Otherwise return serve(the direction with the largest queue; ties broken by order N,E,S,W).

serve(direction)

For each d: if d == direction, skipped[d] = 0; else skipped[d] += 1.

Return direction.

Complexity

Direction count is a constant (4), so each call is O(1) time and O(1) space. Generalised to k approaches:

next_phase is O(k), green_time O(1).

Note: the skipped counter also increments for empty approaches; only those with queues[d] > 0 qualify as starved.

Source: Ex05_Prompt_Chaining_Smart_Traffic/outputs/5_flowchart.md

Step 5 - Flowchart

Chained from 4_algorithm.md (next_phase, serve, green_time).

```
flowchart TD
    A([Start: next_phase queues, emergency]) --> B{Keys of queues equal N,E,S,W?}
    B -- No --> X([Raise ValueError])
    B -- Yes --> C{emergency given?}
    C -- Yes --> D{emergency in N,E,S,W?}
    D -- No --> X
    D -- Yes --> S1[chosen = emergency]
    C -- No --> E[starved = directions with skipped >= max_wait_cycles AND queue > 0]
    E --> F{starved empty?}
    F -- No --> G[chosen = starved direction with largest queue]
    F -- Yes --> H[chosen = direction with largest queue, ties in order N,E,S,W]
    S1 --> I[serve: skipped chosen = 0, all others += 1]
    G --> I
    H --> I
    I --> J[green = green_time queues chosen]
    J --> K{queue x seconds_per_vehicle < min_green?}
    K -- Yes --> L[green = min_green]
    K -- No --> M{queue x seconds_per_vehicle > max_green?}
    M -- Yes --> N[green = max_green]
    M -- No --> O[green = int of queue x seconds_per_vehicle]
    L --> P([Return chosen direction and green seconds])
    N --> P
    O --> P
```

Source:**Ex05_Prompt_Chaining_Smart_Traffic/outputs/7_testing_notes.md**

Step 7 - Testing notes: coverage and uncovered edge cases

The tests themselves live in `tests/test_traffic_controller.py` (6 tests, all passing with `python3 -m unittest discover -s tests`).

What is covered

`green_time` bounds (0, 100, 15), negative queue in `green_time`, longest queue first, emergency pre-emption, starvation (small queue served within 5 cycles), wrong queue keys, invalid `min_green`.

Edge cases NOT covered by the current tests

The behaviours below were checked by hand against the current code (observed results in brackets), but no test asserts them.

All-zero queues: `next_phase` still returns a direction ("N" by tie order) and no "idle" result exists.

Tie-breaking: equal queues resolve in N,E,S,W order; not asserted.

Negative values inside `next_phase` queues are not validated (all -ve queues returned "E", the least negative). Only `green_time` rejects negatives.

Non-integer queue values: `green_time(2.5)` and `green_time(True)` are accepted (both returned 10).

Non-numeric queue values (None, str) and non-dict queues types.

Unknown emergency direction (emergency="X") raising `ValueError`; and lowercase "n".

Emergency does not check queue length: an emergency on an empty approach still gets the green.

Repeated emergencies: skipped counters of other approaches keep growing (after 4 emergencies on W, N and E were both starved).

Starvation exact boundary: served on exactly the `max_wait_cycles`-th skip. Observed pattern with N=50, E=1: N,N,N,E,N,N,N,E.

Starved approach with empty queue must not be picked; starved-tie resolution among several starved approaches.

`max_wait_cycles=0` (every non-empty approach counts as starved, so effectively longest-queue) and negative values are not validated.

`seconds_per_vehicle <= 0` is not validated: with -1, `green_time(5)` returned `min_green` (10).

`max_green < min_green` case (only `min_green=0` is tested).

Exact clamp boundaries (queue 5 -> 10 s, queue 30 -> 60 s) and int truncation for fractional factors.

Controller state persistence across many cycles, and thread safety (no locking; not tested or supported).

Missing tests for the `__main__` demo.

Source:

[Ex05_Prompt_Chaining_Smart_Traffic/outputs/8_documentation.md](#)

Smart Traffic Controller

Step 8 output: chained from the code in `src/traffic_controller.py`, the tests, and `7_testing_notes.md`.

Overview

TrafficController chooses which approach (N, E, S, W) of a 4-way intersection gets the next green and for how long. Priority order: **emergency > starved > longest queue**. Green time is proportional to queue length, clamped between a minimum and maximum.

Install / run

No dependencies (Python 3 standard library).

```
python3 src/traffic_controller.py          # demo
python3 -m unittest discover -s tests      # tests
```

API

- `TrafficController(min_green=10, max_green=60, seconds_per_vehicle=2.0, max_wait_cycles=3)`. Raises `ValueError` if `min_green <= 0` or `max_green < min_green`.
- `green_time(queue_len: int) -> int`: seconds of green, `int(clamp(queue_len * seconds_per_vehicle, min_green, max_green))`. Negative input raises `ValueError`.
- `next_phase(queues: dict, emergency: str | None = None) -> str`: queues must have exactly the keys N, E, S, W. Returns the direction to serve and updates the internal skipped counters. Invalid keys or an unknown emergency direction raise `ValueError`.

Example

```
c = TrafficController()
q = {"N": 12, "E": 3, "S": 8, "W": 1}
d = c.next_phase(q)          # "N"
c.green_time(q[d])          # 24
```

Configuration

Parameter	Default	Meaning
<code>min_green</code>	10	shortest green in seconds
<code>max_green</code>	60	longest green in seconds
<code>seconds_per_vehicle</code>	2.0	green seconds granted per queued vehicle
<code>max_wait_cycles</code>	3	skips after which a non-empty approach is treated as starved

Limitations

- Values inside queues are not validated in `next_phase`; `seconds_per_vehicle` and `max_wait_cycles` are not validated (see `7_testing_notes.md`).
- Always returns a direction, even when all queues are empty.
- No amber/all-red timing, pedestrian phases, or multi-intersection coordination.
- Not thread-safe; state is in memory only.
- The emergency override is unconditional and does not rate-limit repeated requests.

Future work

Input validation hardening, idle result for empty queues, pedestrian phases, persistence of counters, demand forecasting, dashboard integration, and the fail-safe fixed-time fallback described in the architecture.

Source: [Ex05_Prompt_Chaining_Smart_Traffic/prompts/1_problem.md](#)

Define the problem statement for an AI-based smart traffic system for a 4-way urban intersection: current issues, stakeholders, goals and measurable success metrics (avg wait time, throughput, emergency response time). Max 200 words.

Source:

[Ex05_Prompt_Chaining_Smart_Traffic/prompts/2_requirements.md](#)

From the problem below, produce functional and non-functional requirements as a numbered table (ID | Requirement | Priority MoSCoW | Test method).
Problem:
{{previous_output}}

Source:

[Ex05_Prompt_Chaining_Smart_Traffic/prompts/3_architecture.md](#)

Design a system architecture satisfying these requirements: sensors, edge controller, decision engine, signal actuators, cloud dashboard, emergency-vehicle override. Give component list, data flows and tech choices.
Requirements:
{{previous_output}}

Source:

[Ex05_Prompt_Chaining_Smart_Traffic/prompts/4_algorithm.md](#)

Specify the signal-timing algorithm: adaptive green time proportional to queue length with min/max bounds, starvation prevention, and emergency preemption. Provide numbered pseudocode and time complexity.
Architecture:
{{previous_output}}

Source:

[Ex05_Prompt_Chaining_Smart_Traffic/prompts/5_flowchart.md](#)

Turn this algorithm into a Mermaid flowchart (flowchart TD). Only output the mermaid code block.
Algorithm:
{{previous_output}}

Source: [Ex05_Prompt_Chaining_Smart_Traffic/prompts/6_code.md](#)

Implement the algorithm in Python 3 (standard library only): class TrafficController with methods next_phase(queues, emergency=None) and green_time(queue_len). Include type hints and docstrings.
Algorithm:
{{previous_output}}

Source: [Ex05_Prompt_Chaining_Smart_Traffic/prompts/7_testing.md](#)

Write unittest tests for this code: bounds of green time, starvation prevention, emergency preemption, empty queues, invalid input. Then list edge cases not covered.
Code:
{{previous_output}}

Source:

Ex05_Prompt_Chaining_Smart_Traffic/prompts/8_documentation.md

Write README-style documentation: overview, install/run, API, configuration, limitations, future work.
Code and tests:
{{previous_output}}

Appendix: Source Code

Source: Ex05_Prompt_Chaining_Smart_Traffic/src/traffic_controller.py

```
"""Adaptive traffic-signal controller for a 4-way intersection (reference implementation)."""
from typing import Dict, Optional

DIRECTIONS = ("N", "E", "S", "W")

class TrafficController:
    def __init__(self, min_green: int = 10, max_green: int = 60,
                 seconds_per_vehicle: float = 2.0, max_wait_cycles: int = 3):
        if min_green <= 0 or max_green < min_green:
            raise ValueError("invalid green-time bounds")
        self.min_green = min_green
        self.max_green = max_green
        self.seconds_per_vehicle = seconds_per_vehicle
        self.max_wait_cycles = max_wait_cycles
        self._skipped = {d: 0 for d in DIRECTIONS}

    def green_time(self, queue_len: int) -> int:
        """Green seconds proportional to queue length, clamped to [min, max]."""
        if queue_len < 0:
            raise ValueError("queue length cannot be negative")
        return int(max(self.min_green, min(self.max_green, queue_len * self.seconds_per_vehicle)))

    def next_phase(self, queues: Dict[str, int], emergency: Optional[str] = None) -> str:
        """Pick the direction to serve next. Emergency > starved > longest queue."""
        if set(queues) != set(DIRECTIONS):
            raise ValueError("queues must have keys N, E, S, W")
        if emergency is not None:
            if emergency not in DIRECTIONS:
                raise ValueError("unknown emergency direction")
            return self._serve(emergency)
        starved = [d for d in DIRECTIONS if self._skipped[d] >= self.max_wait_cycles and queues[d] > 0]
        if starved:
            return self._serve(max(starved, key=lambda d: queues[d]))
        return self._serve(max(DIRECTIONS, key=lambda d: queues[d]))

    def _serve(self, direction: str) -> str:
        for d in DIRECTIONS:
            self._skipped[d] = 0 if d == direction else self._skipped[d] + 1
        return direction

if __name__ == "__main__":
    c = TrafficController()
    q = {"N": 12, "E": 3, "S": 8, "W": 1}
    d = c.next_phase(q)
    print(d, c.green_time(q[d]), "s")
```

Source:

Ex05_Prompt_Chaining_Smart_Traffic/tests/test_traffic_controller.py

```
import os, sys, unittest
sys.path.insert(0, os.path.join(os.path.dirname(__file__), "..", "src"))
from traffic_controller import TrafficController

class TestTraffic(unittest.TestCase):
    def setUp(self):
        self.c = TrafficController()

    def test_green_bounds(self):
        self.assertEqual(self.c.green_time(0), 10)
        self.assertEqual(self.c.green_time(100), 60)
        self.assertEqual(self.c.green_time(15), 30)

    def test_negative_queue(self):
        with self.assertRaises(ValueError):
            self.c.green_time(-1)

    def test_longest_queue_first(self):
        self.assertEqual(self.c.next_phase({"N": 1, "E": 9, "S": 2, "W": 3}), "E")

    def test_emergency_preempts(self):
        self.assertEqual(self.c.next_phase({"N": 50, "E": 0, "S": 0, "W": 0}, emergency="W"), "W")

    def test_starvation_prevented(self):
        q = {"N": 50, "E": 1, "S": 0, "W": 0}
        served = [self.c.next_phase(q) for _ in range(5)]
        self.assertIn("E", served)

    def test_bad_input(self):
        with self.assertRaises(ValueError):
            self.c.next_phase({"N": 1})
        with self.assertRaises(ValueError):
            TrafficController(min_green=0)

if __name__ == "__main__":
    unittest.main()
```